

Immo2Kin — Vue d'ensemble du projet

Document de synthèse : objectif, architecture, fonctionnement du code et parcours métier.
Pour le détail technique API ou BDD, voir aussi [api.md](#), [architecture.md](#) et [database.md](#).

1. Qu'est-ce que c'est ?

Immo2Kin est une plateforme immobilière orientée Kinshasa (RDC). Elle permet de :

- consulter et rechercher des annonces (liste + carte),
- publier et gérer des biens (propriétaires / agences),
- réserver des logements en location,
- payer (Stripe + Mobile Money simulé),
- échanger via messagerie,
- noter les séjours, demander des vérifications « badge confiance »,
- recevoir des recommandations personnalisées.

Le projet est un monorepo livré en phases (0 à 8 terminées, 75 tests backend passants).

2. Structure du dépôt

```
immo-platform/  
├─ backend/          # API Laravel 12 – cœur métier  
├─ frontend/         # SPA React 19 + Vite + Tailwind  
├─ ai-service/       # Microservice FastAPI (recommandations)  
├─ infra/docker/     # MySQL, Redis, compose local  
├─ docker/           # Scripts déploiement (Railway)  
├─ docs/             # Documentation  
└─ scripts/          # Installation et démarrage dev (PowerShell)
```

Composant	Rôle
backend	Toute la logique métier, la BDD, l'auth, les paiements
frontend	Interface utilisateur, appels API, carte Leaflet
ai-service	Scoring de recommandations (optionnel ; fallback Laravel si indisponible)

En production Railway, une seule image Docker sert l'API Laravel et le frontend compilé sur la même URL (cookies Sanctum simplifiés).

3. Stack technique

Couche	Technologies
API	Laravel 12, PHP 8.2, REST /api/v1
Auth	Laravel Sanctum (SPA cookie + CSRF)
Rôles	Spatie Laravel Permission
Frontend	React 19, Vite 8, Tailwind CSS 4, React Router, Zustand
Carte	Leaflet + coordonnées communes Kinshasa
Temps réel	Laravel Reverb + Echo (dev ; désactivé en MVP Railway)
BDD	MySQL (prod/Docker) ou SQLite (fallback local)
Cache / sessions	Redis (Docker) ou fichier/BDD (fallback)
Paielements	Stripe (+ gateway simulée sans clés) + Mobile Money RDC
Médias	Disque public local ou S3/R2 (MEDIA_DISK)
IA	FastAPI, appel HTTP depuis Laravel

4. Rôles utilisateurs

Quatre rôles (enum UserRole, seedés via Spatie) :

Rôle	Code	Capacités principales
Client	client	Recherche, favoris, réservations, paiements, avis, messagerie
Propriétaire	proprietaire	+ CRUD annonces, photos, réservations reçues, vérification annonce
Agence	agence	Même droits publication que propriétaire
Admin	admin	Panel admin : users, stats, vérifications, sessions actives

Comptes démo (après php artisan db:seed) :

- client@immo.local / password
- proprietaire@immo.local / password
- admin@immo.local / password

5. Architecture backend (clean architecture)

Le backend suit une séparation claire des responsabilités :

```

Requête HTTP
↓
Controller (mince)      → routes/api.php
↓
Form Request (validation) → app/Http/Requests/
↓
Service (logique métier) → app/Services/

```

```

↓
Repository (requêtes BDD) → app/Repositories/
↓
Model Eloquent           → app/Models/
↓
API Resource (JSON)      → app/Http/Resources/

```

Exemple — création d'annonce :

1. PropertyController::store reçoit la requête
2. StorePropertyRequest valide titre, prix, commune, etc.
3. PropertyService::create vérifie le rôle, applique les coordonnées GPS depuis la commune (KinshasaCommuneCoordinates), crée l'enregistrement
4. PropertyRepository persiste en base
5. PropertyResource formate la réponse JSON

Autorisation : politiques Laravel (app/Politiques/) + middleware admin pour les routes /api/v1/admin/*.

Fichiers clés :

Dossier	Contenu
app/Http/Controllers/Api/V1/	~25 contrôleurs REST
app/Services/	Auth, Property, Reservation, Payment, Review, Recommendation...
app/Repositories/	Requêtes complexes (recherche, carte, favoris...)
app/Enums/	Statuts, types, rôles (PHP 8.2 enums)
app/Events/ + Listeners/	Ex. MessageSent pour le temps réel

6. Architecture frontend

SPA React organisée par responsabilité :

```

frontend/src/
├─ pages/           # Écrans (Home, Properties, PropertyDetail, admin...)
├─ components/      # UI réutilisable (PropertyCard, Map, ConfirmDialog...)
├─ layouts/         # MainLayout, AdminLayout
├─ routes/          # AppRouter + gardes (Protected, Owner, Admin, Guest)
├─ services/api/    # Appels HTTP par domaine (properties, auth, payments...)
├─ stores/          # Zustand (authStore)
├─ types/           # Types TypeScript (Property, Reservation...)
├─ config/env.ts    # Variables Vite (API, Reverb, Stripe)
└─ lib/echo.ts      # Laravel Echo (WebSockets)

```

Routing (AppRouter.tsx) :

- Public : /, /properties, /properties/map, /properties/:id

- Auth invité : /login, /register
- Connecté : /dashboard, /favorites, /messages, /reservations
- Propriétaire : /my/properties, création/édition annonces
- Admin : /admin/* (layout séparé)

Client HTTP (services/api/client.ts) : Axios avec withCredentials, cookies CSRF Sanctum, base URL /api en dev (proxy Vite) ou même origine en prod.

État global : authStore (Zustand) pour l'utilisateur connecté ; le reste est chargé par page via les services API.

7. Authentification (Sanctum SPA)

Flux typique :

1. Le frontend appelle GET /sanctum/csrf-cookie (via proxy ou même origine)
2. POST /api/v1/auth/login avec email/mot de passe → cookie de session
3. Requêtes suivantes : cookie + en-tête X-XSRF-TOKEN
4. GET /api/v1/auth/me pour le profil
5. POST /api/v1/auth/logout pour déconnexion

Configuration : SANCTUM_STATEFUL_DOMAINS, CORS_ALLOWED_ORIGINS, FRONTEND_URL (liens e-mail vérification / reset password redirigent vers le SPA).

Limite de débit : throttle:login (5/min par e-mail + IP).

8. Domaines métier — fonctionnement

8.1 Annonces (Property)

- Types : appartement, maison, villa, terrain, bureau, commerce... (PropertyType)
- Statuts : draft, published, archived
- Modes : location ou vente (listing_type)
- Géolocalisation : latitude/longitude déduites de la commune Kinshasa à la création/mise à jour (PropertyService + KinshasaCommuneCoordinates)
- Médias : photos/vidéos via PropertyMediaService, stockées sur le disque configuré (MEDIA_DISK), ordre via sort_order
- Recherche : filtres (prix, commune, type, équipements...), tri, pagination — PropertyRepository
- Carte : endpoint /properties/map — uniquement les annonces avec coordonnées GPS

8.2 Favoris

Table pivot favorites — le client ajoute/retire des annonces publiées.

8.3 Réservations (Reservation)

- Client réserve des dates sur une annonce en location
- Statuts : pending → confirmed / rejected / cancelled
- Calendrier de disponibilité : /properties/{id}/availability
- Le propriétaire confirme ou refuse ; le client peut annuler si pending

8.4 Paiements (Payment)

- Stripe : initiation → confirmation (ou webhook /webhooks/stripe)
- Mobile Money : simulation providers RDC (confirmation manuelle ou auto selon config)
- Sans clés Stripe en local : FakeStripePaymentGateway (tests et démo)
- Paiement possible seulement sur réservation confirmée

8.5 Messagerie (Conversation, Message)

- Un client démarre une conversation sur une annonce publiée (pas sur la sienne)
- Messages stockés en BDD ; diffusion temps réel via Reverb (MessageSent event) en dev
- En production Railway MVP : broadcast en log (pas de WebSocket)

8.6 Avis (Review)

- Note 1–5 + commentaire après un séjour terminé (Reservation completed)
- Un seul avis par client et par annonce

8.7 Vérifications (Verification)

- Le propriétaire peut soumettre une vérification d'annonce
- L'admin approuve ou rejette → badge « Vérifié » côté frontend

8.8 Recommandations

Deux niveaux :

1. Laravel (RecommendationService) : signaux utilisateur (vues, favoris...), annonces populaires, similarité
2. Microservice IA (AiRecommendationClient → ai-service) : scoring hybride si le service répond

Si l'IA est down, le fallback Laravel assure le fonctionnement.

9. API REST — panorama

Toutes les routes sont sous /api/v1 (fichier backend/routes/api.php).

Préfixe / groupe	Exemples
Public	GET /properties, GET /properties/map, GET /health
Auth	POST /auth/login, GET /auth/me
Propriétaire	POST /properties, POST /properties/{id}/images
Client	POST /properties/{id}/reservations, GET /favorites
Paielements	POST /reservations/{id}/payments/stripe
Admin	GET /admin/stats, GET /admin/users

Réponses JSON structurées via API Resources ; erreurs validation avec code: validation_error.

10. Base de données — tables principales

Table	Rôle
users	Comptes + profil
roles / permissions	RBAC Spatie
properties	Annonces
property_images / property_videos	Médias (path, sort_order)
amenities + pivot	Équipements (piscine, parking...)
favorites	Favoris client
reservations	Demandes de location
payments	Paielements liés aux réservations
conversations / messages	Messagerie
reviews	Avis
verifications	Demandes de vérification
recommendation_events	Signaux pour le moteur de reco
sessions	Sessions actives (admin)

Schéma détaillé : database.md.

Seeders : DatabaseSeeder crée rôles, comptes démo, annonces (PropertySeeder, ExtraPropertySeeder), réservations, conversations, etc.

11. Microservice IA (ai-service/)

- FastAPI, Python 3.12
- Endpoints : GET /health, recommandations (router recommendations)
- Appelé par Laravel via AI_SERVICE_URL + clé AI_SERVICE_API_KEY
- Docker séparé en local (infra/docker) ; service Railway optionnel (root ai-service/)

12. Médias et stockage

- Upload : PropertyMediaService → chemin properties/{id}/images/{uuid}.ext
- URL publique : PropertyImage::url() via MediaStorage (disque public ou s3)
- Local : storage/app/public/ + lien symbolique public/storage
- Cloud : Cloudflare R2 / AWS S3 (MEDIA_DISK=s3)
- Export / import : commandes Artisan property-media:export, property-media:import (manifeste JSON + sort_order)

Guide : media-storage.md.

13. Développement local

Démarrage rapide (Windows) :

```
.\scripts\start-dev.ps1 -Docker
```

Service	URL
Frontend	http://localhost:5173
API	http://localhost:8000
Health	http://localhost:8000/api/v1/health

Tests :

```
cd backend
php artisan test
```

Variables frontend : laisser VITE_API_URL vide en dev → proxy Vite vers l'API (cookies CSRF fiables).

14. Déploiement (aperçu)

- Railway : Dockerfile racine, MySQL plugin, variables d'environnement — railway.md
- Images : R2/S3 recommandé — media-storage.md
- Limitations MVP Railway : pas de Reverb, pas de Redis, e-mails en log

15. Où chercher quoi dans le code ?

Besoin	Fichier / dossier
Ajouter une route API	backend/routes/api.php
Logique création annonce	PropertyService.php, PropertyController.php
Filtres recherche / carte	PropertyRepository.php
Coordonnées communes	KinshasaCommuneCoordinates.php
Upload photos	PropertyMediaService.php, PropertyMediaController.php
Paieement Stripe	PaymentController.php, StripePaymentGateway.php
Page liste annonces	frontend/src/pages/PropertiesPage.tsx
Carte Leaflet	frontend/src/components/PropertyMap.tsx
Gardes routes React	frontend/src/routes/ProtectedRoute.tsx, OwnerRoute.tsx, AdminRoute.tsx
Config API frontend	frontend/src/config/env.ts
Seed données démo	backend/database/seeder/

16. Principes de conception retenus

1. Contrôleurs minces — la logique vit dans les Services
2. Validation centralisée — Form Requests, pas de règles dans les contrôleurs
3. API cohérente — préfixe v1, Resources JSON, codes d'erreur explicites
4. SPA même origine en prod — évite les problèmes CORS/Sanctum
5. Fallbacks — SQLite/file sans Docker ; FakeStripe sans clés ; reco Laravel sans IA
6. Kinshasa-first — communes, coordonnées, Mobile Money, contexte RDC

17. Documentation complémentaire

Document	Sujet
architecture.md	Schéma technique court
api.md	Endpoints REST détaillés
database.md	Schéma tables
railway.md	Hébergement production
media-storage.md	Photos cloud + import
README.md	Installation, tests, GitHub

Document généré pour présentation et reprise du projet — Immo2Kin, plateforme immobilière Kinshasa.